Assignment No: 14

Title: Develop a Node.js and Express application to create a web server, implement routing and serve static resources.

Aim: To develop a server-side application using Node.js and the Express framework that creates an HTTP web server, implements routing including route parameters and query strings, serves static resources from a public directory, and handles errors with middleware.

Objectives:

1. To understand Node.js, its event-driven non-blocking model and the npm ecosystem.
2. To create an HTTP server using the built-in http module and then using Express.
3. To initialise a project with package.json and install Express.
4. To implement routing for different paths and HTTP methods.
5. To read route parameters, query-string values and a JSON request body.
6. To serve static resources using the express.static middleware.
7. To write custom middleware and understand the next() function.
8. To handle 404 and 500 responses with dedicated middleware.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- Node.js (v18 or later) and npm
- Express 5.x
- Postman or curl (for testing routes)

Theory:

1. Introduction to Node.js

Node.js is a runtime that executes JavaScript outside the browser, built on the V8 engine. It is single-threaded and event-driven: instead of blocking a thread while waiting for input or output, it registers a callback and continues. This makes it well suited to applications that handle many concurrent connections with little computation per request.

npm, the Node package manager, installs third-party packages into node_modules and records them in package.json.

2. A Server Using the Built-in http Module

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js');
});

server.listen(3000, () => console.log('Listening on 3000'));
```

This works but leaves routing, body parsing, static files and error handling to be written by hand, which is why a framework is normally used.

3. Express

Express is a minimal web framework for Node.js. It supplies a router, a middleware pipeline, request and response helpers and static file serving.

Method	Purpose
app.get(path, handler)	Handles a GET request for the path
app.post / put / patch / delete	Handles the other HTTP verbs
app.use(fn)	Registers middleware for every request
app.use(path, router)	Mounts a router under a path prefix
app.listen(port, cb)	Starts the server

4. Routing, Parameters and Query Strings

Source	Example URL	Read in Express as
Route parameter	/manuscripts/LS-0101	req.params.accession
Query string	/search?script=Modi&page=2	req.query.script, req.query.page
JSON body	POST /manuscripts	req.body (needs express.json())
Header	any request	req.headers['x-api-key']

Routes are matched in the order they are registered, so a specific route must be declared before a more general one that would also match.

5. Middleware

Middleware is a function with the signature (req, res, next). It can inspect or modify the request and response, end the response, or call next() to pass control to the next function in the chain. Error-handling middleware takes four arguments, (err, req, res, next), and Express recognises it by that arity.

- express.json() parses a JSON request body into req.body.
- express.urlencoded({ extended: true }) parses an HTML form submission.
- express.static(dir) serves files from a directory.
- A custom logger records the method, URL and duration of each request.

6. Serving Static Resources

express.static maps a directory onto a URL prefix. With app.use(express.static('public')), the file public/style.css is served at /style.css. With app.use('/assets', express.static('public')) it is served at

/assets/style.css instead. Static middleware is registered before the routes so that an existing file is returned without entering the router.

7. Status Codes

Code	Meaning
200 OK	The request succeeded
201 Created	A resource was created
400 Bad Request	The request was malformed or failed validation
404 Not Found	No route or resource matched
500 Internal Server Error	An unhandled error occurred on the server

---

Course Project Implementation:

Course-Project Mapping: Not available in the course project. LipiSutra has no backend of its own: it is a pure client-side React application that calls Google's APIs directly from the browser, and the only server involved is the Vite development server (vite.config.js, package.json script "dev": "vite"), which is a build tool, not application code. The project does contain an nginx.conf and a Dockerfile that serve the built static files in production, which is the deployment-level equivalent of express.static, but there is no Node.js or Express source file to submit. A complete Express server is therefore supplied below, written as the backend LipiSutra would need in order to move its API keys off the client.

Requirement	In project?	Nearest equivalent
Node HTTP server	No	Vite dev server (a build tool, not app code)
Express routing	No	client-side view switching in src/App.jsx
Static file serving	Config only	nginx.conf try_files + Dockerfile
Middleware pipeline	No	-

Program:

File 1: package.json

```
{
  "name": "lipisutra-server",
  "version": "1.0.0",
  "type": "commonjs",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev":   "node --watch server.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

File 2: server.js - the Express web server

```
const express = require("express");
const path    = require("path");

const app  = express();
const PORT = process.env.PORT || 3000;

/* ---------------------------------------------------------------
   1. BUILT-IN MIDDLEWARE
   --------------------------------------------------------------- */
app.use(express.json());                                // parse JSON bodies
app.use(express.urlencoded({ extended: true }));      // parse HTML form posts

/* ---------------------------------------------------------------
   2. CUSTOM MIDDLEWARE : a request logger
      Runs for every request, then calls next() to continue.
   --------------------------------------------------------------- */
app.use((req, res, next) => {
  const started = Date.now();
  res.on("finish", () => {
    console.log(`${req.method} ${req.originalUrl} -> ${res.statusCode} ` +
                `(${Date.now() - started} ms)`);
  });
  next();
});

/* ---------------------------------------------------------------
   3. STATIC RESOURCES
      public/index.html  -> http://localhost:3000/
      public/style.css   -> http://localhost:3000/style.css
      public/hero.png    -> http://localhost:3000/assets/hero.png
   --------------------------------------------------------------- */
app.use(express.static(path.join(__dirname, "public")));
app.use("/assets", express.static(path.join(__dirname, "public")));

/* ---------------------------------------------------------------
   4. IN-MEMORY DATA (a database replaces this in Assignment 15)
   --------------------------------------------------------------- */
const manuscripts = [
  { accession: "LS-0101", title: "Modi Land Grant",          script: "Modi",
    year: 1712, confidence: 92.5, status: "verified" },
  { accession: "LS-0102", title: "Devanagari Verse Folio",   script: "Devanagari",
    year: 1588, confidence: 87.0, status: "verified" },
  { accession: "LS-0103", title: "Brahmi Rock Inscription",  script: "Brahmi",
    year: -230, confidence: 71.25, status: "pending" }
];

/* ---------------------------------------------------------------
   5. ROUTING
   --------------------------------------------------------------- */

// 5a. a simple route returning HTML
app.get("/about", (req, res) => {
  res.send(`<h1>LipiSutra</h1>
            <p>Server time: ${new Date().toLocaleString()}</p>
            <p><a href="/">Back to the home page</a></p>`);
});

// 5b. a route returning JSON
app.get("/api/health", (req, res) => {
  res.json({ status: "ok", uptimeSeconds: Math.round(process.uptime()) });
});

// 5c. QUERY STRING   /search?script=Modi&verified=true
app.get("/search", (req, res) => {
  const { script, verified } = req.query;
  let rows = manuscripts;

  if (script) {
    rows = rows.filter(m => m.script.toLowerCase() === script.toLowerCase());
```

```
  }
  if (verified === "true") {
    rows = rows.filter(m => m.status === "verified");
  }

  res.json({ query: req.query, count: rows.length, results: rows });
});

// 5d. ROUTE PARAMETER   /manuscripts/LS-0101
//     registered AFTER /search so the literal path is not shadowed
app.get("/manuscripts/:accession", (req, res) => {
  const found = manuscripts.find(
    m => m.accession.toUpperCase() === req.params.accession.toUpperCase()
  );

  if (!found) {
    return res.status(404).json({ error: `No manuscript ${req.params.accession}` });
  }
  res.json(found);
});

// 5e. a POST route reading a JSON body
app.post("/manuscripts", (req, res) => {
  const { accession, title, script, year } = req.body;

  if (!accession || !title || !script) {
    return res.status(400).json({ error: "accession, title and script are
required" });
  }

  const record = { accession, title, script, year: Number(year) || 0,
                   confidence: 0, status: "pending" };
  manuscripts.push(record);
  res.status(201).json(record);
});

// 5f. a route that deliberately throws, to exercise the error handler
app.get("/boom", (req, res) => {
  throw new Error("Simulated server failure");
});

/* ------------------------------------------------------------
   6. 404 HANDLER - reached only if no route above matched
   ------------------------------------------------------------ */
app.use((req, res) => {
  res.status(404).send(`<h1>404 - Not Found</h1>
                        <p>No route for ${req.method} ${req.originalUrl}</p>`);
});

/* ------------------------------------------------------------
   7. ERROR HANDLER - four arguments, so Express treats it as one
   ------------------------------------------------------------ */
app.use((err, req, res, next) => {
  console.error("Unhandled error:", err.message);
  res.status(500).json({ error: "Internal Server Error", detail: err.message });
});

/* ------------------------------------------------------------
   8. START
   ------------------------------------------------------------ */
app.listen(PORT, () => {
  console.log(`LipiSutra server listening on http://localhost:${PORT}`);
});
```

File 3: public/index.html - a static resource served by express.static

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>LipiSutra Server</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>LipiSutra Node Server</h1>
  <p>This page, its stylesheet and its image are all served statically
     from the <code>public/</code> directory.</p>

  <img src="/assets/hero.png" alt="Manuscript" width="320">

  <h2>Available routes</h2>
  <ul>
    <li><a href="/about">/about</a> - HTML response</li>
    <li><a href="/api/health">/api/health</a> - JSON response</li>
    <li><a href="/search?script=Modi">/search?script=Modi</a> - query string</li>
    <li><a href="/manuscripts/LS-0101">/manuscripts/LS-0101</a> - route parameter</li>
    <li><a href="/boom">/boom</a> - triggers the error handler</li>
    <li><a href="/nothing-here">/nothing-here</a> - triggers the 404 handler</li>
  </ul>
</body>
</html>
```

File 4: public/style.css - a static stylesheet

```
body { font-family: Georgia, serif; background: #f5ede0; color: #1a0800;
       max-width: 760px; margin: 40px auto; padding: 0 20px; }
h1   { color: #6B2D0E; }
code { background: #ede0cc; padding: 2px 5px; border-radius: 3px; }
img  { border: 1px solid #8B4513; border-radius: 8px; }
a    { color: #6B2D0E; }
```

Commands used

```
# create the project
mkdir lipisutra-server && cd lipisutra-server
npm init -y

# install the framework
npm install express

# create public/ and the files above, then start the server
node server.js
# or, restarting automatically on every save:
node --watch server.js

# test the routes from a second terminal
curl http://localhost:3000/api/health
curl "http://localhost:3000/search?script=Modi&verified=true"
curl http://localhost:3000/manuscripts/LS-0101
curl -X POST http://localhost:3000/manuscripts \
     -H "Content-Type: application/json" \
     -d '{"accession":"LS-0104","title":"Sharada
Folio","script":"Sharada","year":1104}'
```

Output:

- node server.js prints "LipiSutra server listening on http://localhost:3000".
- Opening http://localhost:3000/ serves public/index.html, and the browser then loads /style.css and /assets/hero.png - all three handled by express.static without entering the router.
- /about returns an HTML page showing the current server time.

- /api/health returns {"status":"ok","uptimeSeconds":12}.
- /search?script=Modi&verified=true returns the echoed query object, count 1 and the Modi Land Grant record.
- /manuscripts/LS-0101 returns that single record read from req.params; /manuscripts/LS-9999 returns HTTP 404 with {"error":"No manuscript LS-9999"}.
- The POST with a JSON body returns HTTP 201 and the created record; a POST missing the script field returns HTTP 400 with the validation message.
- /boom is caught by the four-argument error middleware and returns HTTP 500 with {"error":"Internal Server Error","detail":"Simulated server failure"}.
- /nothing-here falls through every route to the 404 middleware and returns the Not Found page.
- The custom logger prints one line per request, for example "GET /search?script=Modi -> 200 (3 ms)".

Conclusion:

In this assignment, we developed a Node.js and Express application that creates a web server, implements routing and serves static resources. We registered routes for several HTTP verbs, read data from route parameters, query strings and a JSON request body, served a static directory with express.static, wrote custom logging middleware that calls next(), and added dedicated 404 and 500 handlers, observing the correct status code for each case. We also established that the LipiSutra course project has no backend of its own - its static files are served by nginx in production - so this server was written specifically for the assignment.

References:

9. Node.js Documentation, https://nodejs.org/docs/latest/api/
10. Express 5.x API Reference, https://expressjs.com/en/5x/api.html
11. MDN Web Docs - Express/Node introduction, https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction
12. Express - Serving static files, https://expressjs.com/en/starter/static-files.html
13. Express - Using middleware, https://expressjs.com/en/guide/using-middleware.html